

122. Un son pour chaque action

Une note quand on ramasse, un « toc » quand on se cogne : le joueur entend ce qui se passe.

1. Ce qu'il faut comprendre

Dans un jeu, le son **dit** ce qui arrive, avant même qu'on le voie : la pièce ramassée tinte, le mur fait « toc ». Ce programme reprend le mur de la leçon « lire la case devant soi » et la pièce de « deux boîtes qui se touchent », et donne à chacun son son.

Ce qui est nouveau ici : `bruit(duree, volume)`. Il frappe sur la **voix 4**, celle du bruit : pas une note, un choc.

`bruit(4, 10)` : une frappe très courte (durée 4), au volume 10. Un troisième argument, le **grain**, le rend plus sourd ou plus sifflant.

La pièce joue sur la **voix 1**, le mur sur la **voix 4** : ils peuvent sonner **en même temps** sans se couper. Deux sons sur la même voix, c'est le second qui remplace le premier.

Le « toc » n'est joué qu'**une fois par choc**. Tant qu'on pousse contre le mur, `bloque` reste vrai ; on ne joue le bruit qu'au **front** de `bloque` : vrai maintenant, faux à l'image d'avant. C'est le même front que pour un bouton — il marche pour **n'importe quelle** condition.

À toi : donne au mur de gauche un grain différent (`bruit(4, 10, 3)`) : les deux murs ne sonnent plus pareil.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <Tuile>` → [0.107.2. Tuile — dessiner une case](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <bruit>` → [0.83.1. La fonction bruit\(\) — un bruit](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <sprite>` → [0.69.4. La fonction sprite\(\) — un lutin au pixel près](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1139 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	82 o	← la console, à l'allumage — la mise en route de la console
main()	362 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
AdresseCase	10 o	← lire() — trouve l'adresse d'une case de l'écran

```

EcrireTexte      10 o ← texte()  – écrit un texte, lettre par lettre
EcrireNombre     33 o ← nombre() – écrit un nombre en chiffres
RangerLutins     11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles     76 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte     14 o ← démarrage – vide les deux cartes de l'écran
LireManette      37 o ← bouton() – lit les huit boutons
Tuiles           288 o ← CopierTuiles – 15 lettres sur 43, et 3 tuiles dessinées
DonneesTransfert 10 o ← InstallerTransfert – la petite routine qui copie les lutins
Notes            120 o ← note()  – la table des notes de musique
"SCORE"          5 o ← texte()  – un texte gravé
PX_0             2 o ← sprite()  – une table gravée

```

Lettres gravées : CEORS0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, Hasard, EffacerFond, EffacerPanneau

Fonctions du langage appelées :

```

note()   × 1   56 o sur place, dans main() + Notes (120 o)
sprite() × 2   54 o sur place, dans main() + PX_0 (2 o)
bruit()  × 1   39 o sur place, dans main()
lire()   × 2   38 o sur place, dans main() + AdresseCase (10 o)
nombre() × 2   20 o sur place, dans main() + EcrireNombre (33 o)
poser()  × 2   12 o sur place, dans main()
texte()  × 1   11 o sur place, dans main() + EcrireTexte (10 o)
bouton() × 2   10 o sur place, dans main() + LireManette (37 o)
image()  × 1    6 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : un héros va à gauche et à droite entre deux murs.
// Il ramasse une pièce (« ding ! ») et se cogne aux murs (« toc ! »).
//
// Ce qui est nouveau : bruit(durée, volume) : un choc sur la voix 4, celle
// du bruit (pas une note). bruit(4, 10) = une frappe très courte, volume 10.
// La pièce sonne sur la voix 1, le mur sur la voix 4 : ils peuvent sonner
// en même temps sans se couper.

```

```

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <nombre>   // écrit un nombre en chiffres
#include <poser>    // pose une tuile sur une case du fond
#include <bouton>   // lit un bouton de la manette
#include <lire>     // lit la tuile posée sur une case
#include <bruit>    // joue un bruit
#include <note>     // joue une note
#include <sprite>   // place un lutin de 8 × 8 au pixel près

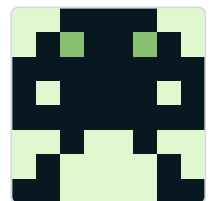
```

2. UN DESSIN — HEROS

```

Tuile HEROS = {
    "..###..",
    ".#-##-#.",
    "#####",
    "#.####.#",
    "#####",
    "..#..#..",
    ".#...#.",
    "##....##",
};

```



à l'écran

3. UN DESSIN — PIECE

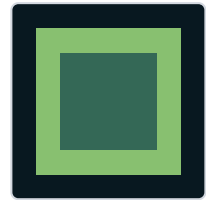
```
Tuile PIECE = {  
    "..####..", ".#---#.", "#-#-#-#", "#-#-#-#",  
    "#-#-#-#", "#-#-#-#", ".#---#.", "..####..",  
};
```



à l'écran

4. UN DESSIN — MUR

```
Tuile MUR = {  
    "33333333", "31111113", "31222213", "31222213",  
    "31222213", "31222213", "31111113", "33333333",  
};
```



à l'écran

5. UNE TABLE GRAVÉE DANS LA CARTOUCHE — PX

```
const uint8_t PX[] = { 100, 48 }; // les deux places de la pièce (x en pixels)
```

6. UNE VARIABLE — X

```
uint8_t x = 60; // le héros, en pixels
```

7. UNE VARIABLE — I

```
uint8_t i = 0; // la place de la pièce : 0 ou 1
```

8. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

9. UNE VARIABLE — AVANTBLOQUE

```
uint8_t avantBloque = 0; // poussait-on contre un mur à l'image d'avant ?
```

10. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {                // Le jeu commence ici.
    texte(1, 1, "SCORE");
    nombre(7, 1, score);
    poser(3, 8, MUR);        // un mur en colonne 3 (pixels 24 à 31)
    poser(16, 8, MUR);       // un mur en colonne 16 (pixels 128 à 135)

    while (true) {           // La boucle du jeu :
        image();

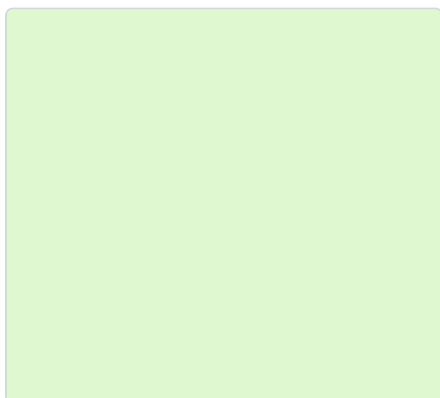
        uint8_t bloque = 0;   // 1 si on pousse contre un mur à cette image
        if (bouton(DROITE)) {
            if (lire((x + 8) / 8, 8) != MUR) x++; else bloque = 1;    // libre : on avance ;
        }                                                              // sinon : bloqué.
        if (bouton(GAUCHE)) {
            if (lire((x - 1) / 8, 8) != MUR) x--; else bloque = 1;
        }
        if (bloque && !avantBloque) {    // Le FRONT de « bloque » : bloqué maintenant,
            // pas avant → le tout début du choc.
            bruit(4, 10);                // toc : une fois par choc, sur la voix 4
        }
        avantBloque = bloque;

        if (x < PX[i] + 8 && PX[i] < x + 8) {    // la pièce est sur la même ligne :
            score++;                            // la largeur suffit
            i = 1 - i;                          // l'autre place : 0 → 1 → 0 ...
            nombre(7, 1, score);
            note(1, MI5, 8, 12);                // ding : sur la voix 1
        }

        sprite(0, x, 64, HEROS);               // le héros (ligne de pixels 64 = ligne de cases 8)
        sprite(1, PX[i], 64, PIECE);           // la pièce
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



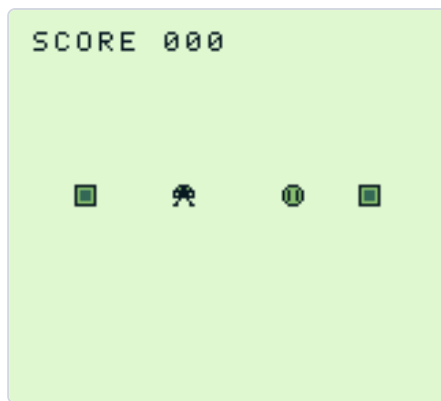
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros entre deux murs et une pièce ; la pièce tinte quand on la ramasse, le mur fait « toc » à chaque choc.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ la pièce ramassée tinte sur la voix 1 (MI5, vers 659) (659)
- ✓ le mur fait toc sur la voix 4
- ✓ un seul toc, même en poussant encore

Ensuite : **123. Un petit air qui joue tout seul** — chapitre 21 : Le son : notes, bruits, airs. · gameboy3 — écrire une cartouche Game Boy en C++